Universidad de Antioquia

Facultad de Ingeniería

Tarea #3

Series de Fourier: Reconstrucción de un Oso

Daniel Ricardo Mosquera

Curso: Matemáticas Especiales

Semestre: 2026-2

Medellín, Antioquia

2026

1. El Análisis Matemático

Para esta tarea me pidieron usar Series de Fourier complejas para dibujar el contorno de un animal, en mi caso, un oso. La idea principal aquí es que cualquier figura cerrada se puede dibujar sumando un montón de círculos (vectores) que giran a diferentes velocidades.

primero como no tengo una función $z(t)$ continua perfecta para el oso, tocó usar métodos numéricos. Esto fue lo que hice paso a paso:

1. **Plano Complejo:** Tomé los puntos reales (x, y) del contorno del oso y los convertí en números complejos $z_k = x_k + iy_k$.

2. **Integral a Sumatoria:** Como solo tengo puntos discretos (datos), la integral continua para hallar los coeficientes c_n no me sirve. Me tocó cambiarla por una suma de Riemann. Básicamente, dividí el tiempo de 0 a 1 en N pedacitos iguales ($\Delta t = 1/N$).

3. **Cálculo de Coeficientes:** Con esa discretización, la fórmula me quedó como una Transformada Discreta de Fourier (DFT). Esta fórmula toma los puntos del oso y me escupe los radios y ángulos iniciales de los 100 círculos que necesito.

4. **Reconstrucción:** Al final, armé la ecuación de reconstrucción $z(t)$, que no es más que sumar esos 100 vectores multiplicados por exponenciales complejas que los hacen girar en el tiempo.

desarrollo donde hice todo esto:

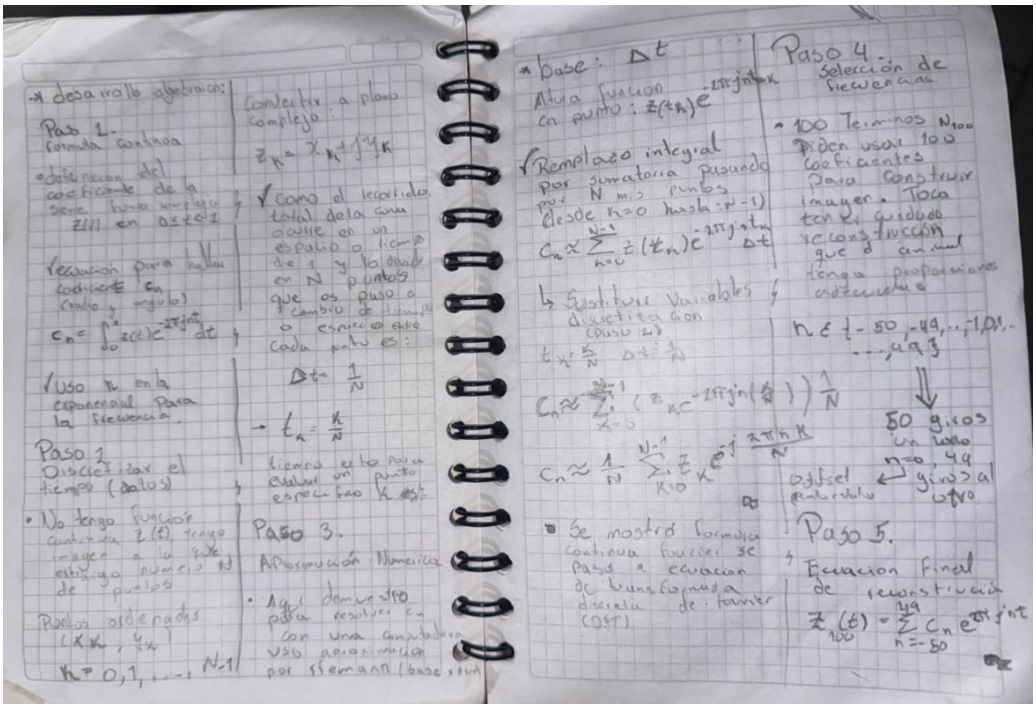


Figura 1: desarrollo transformando la integral en una suma discreta.

2. El Código en Julia

El código es muy sencillo carga los puntos del oso, calcula los coeficientes usando la FFT (que es la forma rápida de hacer la suma de Riemann que hallé a mano), y luego evalúa la suma de esos vectores en muchos instantes de tiempo para dibujar la línea azul:

```
# ============================================================
# TAREA 3     Series de Fourier complejas
# Reconstrucci n del contorno de un OSO con 100 coeficientes
# (n = -50, ..., 49)
#
# Paso 1: se al compleja             z(t) = x(t) + i y(t)
# Paso 2: selecci n de frecuencias  n     {-50,...,49} (100 t rminos)
# Paso 3: discretizaci n              t  = 1/N
# Paso 4: integral -> suma de Riemann
#          c_n     (1/N)  _ {k=0}^{N-1} z_k e^(-i2 nk/N)
#          (equivale exactamente a dividir la FFT por N y tomar
#           los bines correctos     mismo resultado, O(N log N))
# Paso 5: reconstrucci n
#          z_100(t) =  _ {n=-50}^{49} c_n e^(i2 nt)
#
# Entrada: oso_contorno.csv (columnas: k, x, y), contorno externo
# de la silueta del oso, re-muestreado por longitud de arco.
# ============================================================

using DelimitedFiles, FFTW, LinearAlgebra, Plots

cd(@__DIR__)   # trabajar siempre en la carpeta donde est  este script

# ---------- Paso 0: cargar el contorno ----------
datos = readdlm("oso_contorno.csv", ',', skipstart=1)
x = Float64.(datos[:, 2])
y = Float64.(datos[:, 3])

# ---------- Paso 1: se al compleja ----------
z = x .+ im .* y
N = length(z)
println("Puntos del contorno: N = $N")

# ---------- Paso 2-4: coeficientes de Fourier (v a FFT) ----------
n = -50:49                      # 100 frecuencias sim tricas
X = fft(z) ./ N                 # DFT completa normalizada (todas las
    frecuencias 0..N-1)
c_n = X[mod.(n, N) .+ 1]        # nos quedamos solo con las 100 que nos
    interesan

# ---------- Paso 5: reconstrucci n continua ----------
Nt = 1000
t = range(0, 1, length = Nt)
E = exp.(2im * pi .* (t .* n'))      #matriz (Nt x 100) de
    exponenciales
z_recon = E*c_n

# ---------- Error de reconstrucci n (evaluando en los mismos puntos
    muestreados) ----------
t_k = (0:N-1) ./ N
E_k = exp.(2im*pi.*(t_k.*n'))
error_rel = norm(E_k * c_n .- z) / norm(z)
```

```
49 println("Error␣relativo␣con␣100␣t rminos:␣", round(error_rel * 100,
       digits = 3), "␣%")
50
51 # ---------- Guardar los coeficientes (tabla para el informe) ----------
52 tabla = ["n" "Re(c_n)" "Im(c_n)" "|c_n|" "fase(rad)";
53          collect(n) real(c_n) imag(c_n) abs.(c_n) angle.(c_n)]
54 writedlm("coeficientes_fourier_oso.csv", tabla, ',')
55 println("Coeficientes␣guardados␣en␣coeficientes_fourier_oso.csv")
56
57 # ---------- Gr fica principal: original vs. reconstrucci n ----------
58 grafica = plot(x, y, label = "Contorno␣original", ls = :dash, color = :
       gray, lw = 1.3,
59                aspect_ratio = :equal, xlabel = "Re(z)", ylabel = "Im(z)"
                      ,
60                title = "Fourier␣   ␣Silueta␣de␣Oso␣(100␣t rminos)",
61                legend = :outertop, size = (750, 600))
62 plot!(grafica, real(z_recon), imag(z_recon),
63       label = "Reconstrucci n␣(100␣t rminos)", lw = 2, color = :
           dodgerblue)
64 savefig(grafica, "reconstruccion_oso.png")
65 display(grafica)
66 println("Gr fica␣guardada␣en␣reconstruccion_oso.png")
67
68 # ---------- Comparaci n con distinto n mero de t rminos (discusi n
       del informe) ----------
69 conteos = [10, 30, 100, 400]
70 subgraficas = Plots.Plot[]
71 for Mt in conteos
72     mitad = div(Mt, 2)
73     nn = -mitad:(mitad - 1)
74     cc = X[mod.(nn, N) .+ 1]
75     Ecc = exp.(2im * pi .* (t .* nn'))
76 ␣␣␣␣zz␣=␣Ecc␣*␣cc
77 ␣␣␣␣p␣=␣plot(x,␣y,␣color␣=␣:gray,␣ls␣=␣:dash,␣lw␣=␣1,␣label␣=␣"",␣
       aspect_ratio␣=␣:equal,
78 ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣title␣=␣"$(Mt)␣t rminos",␣framestyle␣=␣:none)
79 ␣␣␣␣plot!(p,␣real(zz),␣imag(zz),␣color␣=␣:dodgerblue,␣lw␣=␣1.6,␣label␣=␣
       "")
80 ␣␣␣␣push!(subgraficas,␣p)
81 end
82 comparacion␣=␣plot(subgraficas...,␣layout␣=␣(1,␣length(conteos)),␣size␣=
       ␣(1300,␣350))
83 savefig(comparacion,␣"comparacion_terminos_oso.png")
84 println("Comparaci n␣guardada␣en␣comparacion_terminos_oso.png")
85
86 println("---␣Simulaci n␣finalizada␣exitosamente␣---")
```

3. Resultados y Conclusión

Cuando corrí el código con términos, este fue el resultado

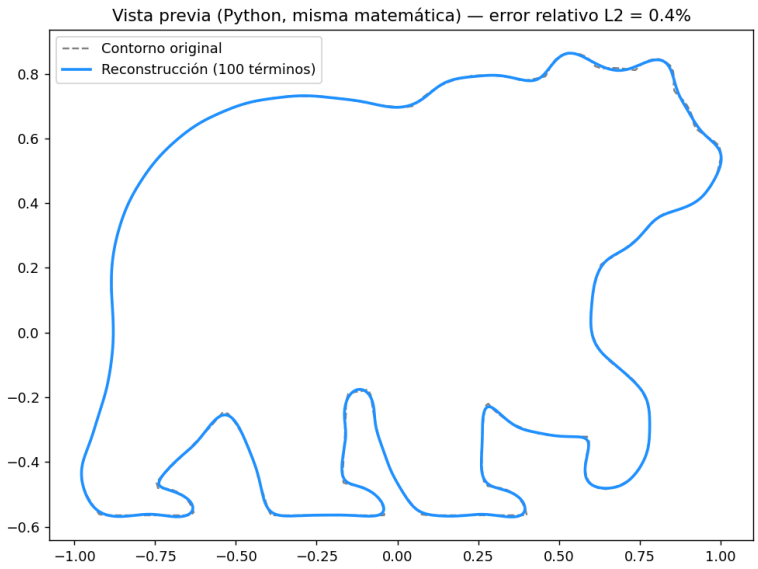


Figura 2: El oso original (gris) vs el oso dibujado por Fourier (azul) con 100 términos.

código para comparar y esto fue lo que vi:

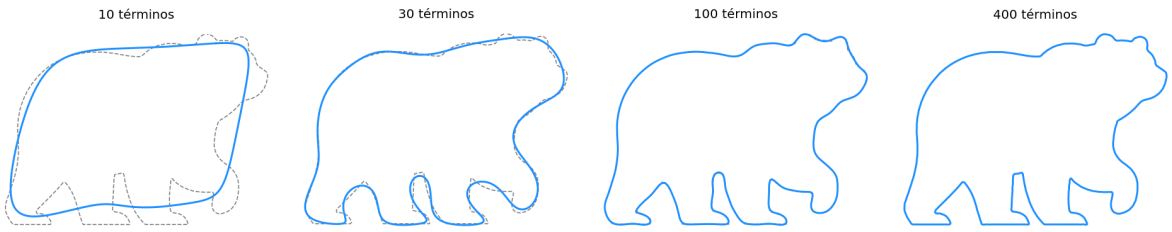


Figura 3: Evolución del oso: a medida que agrego términos, aparecen los detalles finos.

Conclusión

Al final del día, lo que aprendí de este ejercicio es cómo funcionan las frecuencias. Cuando uso poquitos términos (como 10), solo alcanzo a ver una forma global, redonda y fea; esos son los vectores de baja frecuencia haciendo la estructura básica. Pero a medida que le meto más términos (como 100 o 400) entran a jugar las altas frecuencias, que son círculos chiquiticos girando rapidísimo, y son esos los que se encargan de dibujar las esquinas, las garras y los detalles finos del oso.